Pytex.school

От вайб-кодинга к профессиональной AI-разработке

Итоговый синтез: Инженерный харнес, Оркестрация конвейера, База знаний и GSD

Алексей Жиряков // Pytex.school
Pytex.school

ПРЕПОДАЮ Я, АЛЕКСЕЙ ЖИРЯКОВ, ТЕХ. РУКОВОДИТЕЛЬ В СБЕРБАНКЕ

15+ лет в backend-разработке, Техлид инженерных команд
Член программного комитета Highload

Сбербанк	Техлид	Отвечаю за работу с данными для ИИ на платформе ГенИИ
KION	CTO направления	Построил highload-архитектуру, платформу данных и персональные витрины
MTC Web Services	Техлид	Запустил AI-агентов для автотестов и ускорения код-ревью



3 ключевые боли вайб-кодинга: что ломается в linksvc

1. Боль 1. Бесконечный цикл правок (Fix Loop of Death)

   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

3 ключевые боли вайб-кодинга: что ломается в linksvc

1. Боль 1. Бесконечный цикл правок (Fix Loop of Death)

   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

2. Боль 2. Каскадный разрыв контракта (DTO)

   Самовольный рефакторинг полей ломает внешние сервисы и клиентов.

3 ключевые боли вайб-кодинга: что ломается в linksvc

1. **Боль 1. Бесконечный цикл правок (Fix Loop of Death)**
   Агент рапортует «Всё готово!», но ходит по кругу, срезает assert и прячет сбои.

2. **Боль 2. Каскадный разрыв контракта (DTO)**
   Самовольный рефакторинг полей ломает внешние сервисы и клиентов.

3. **Боль 3. Архитектурный дрейф и слом конвенций**
   Агент нарушает границы слоёв, тащит чужой стек и перекраивает чужой код.

Ловушка «среднего Python» (The Average Python Trap)

Синхронный вызов в async

`requests.get(url)`

Блокирует event loop FastAPI под нагрузкой.

Дата без часового пояса

`datetime.now()`

Наивное время ломает сравнение с UTC в базе.

Подавление исключений

`except Exception: pass`

Скрывает сбой БД и коллизии, клиент видит ложный 200 OK.

Модель повторяет средний открытый код. Отсюда и архитектурный дрейф: чужие библиотеки и стиль вместо конвенций команды.

Лестница принуждения: от надежды на модель к детерминированному барьеру

Скил сам по себе — текст, а не замок

Вероятностная природа: модель срезает углы в длинном диалоге

Тестовый судья: агент переписывает assert под свой баг

Тавтологичные проверки: assert True вместо реальной логики

Выборочное послушание: пропуск стоп-условий ради 200 OK

Лестница принуждения: от надежды на модель к детерминированному барьеру

Скил сам по себе — текст, а не замок

Вероятностная природа: модель срезает углы в длинном диалоге

Тестовый судья: агент переписывает assert под свой баг

Тавтологичные проверки: assert True вместо реальной логики

Выборочное послушание: пропуск стоп-условий ради 200 OK

Внешнее детерминированное принуждение

Характеризационные тесты: текущее поведение зафиксировано до правок

PreToolUse хуки: физический запрет записи в tests/ и конфиги

Mutation Drill: тест обязан падать при контролируемой поломке

Серверный CI: ruff + basedpyright + pytest как детерминированный рубеж

ПЛАН ИТОГОВОГО ЗАНЯТИЯ

9 **01. СКИЛЫ, ХАРНЕСЫ И МАШИННЫЕ ГАРДРАЙЛЫ**

9 **02. ОРКЕСТРАЦИЯ КОНВЕЙЕРА И ПРИЁМКА РАЗНЫМИ**

9 **03. БАЗА ЗНАНИЙ КАРПАТИ И МЕТОДИКА GSD**

Итоговый синтез трёх дней: от хаотичного чата и ловушки среднего Python к надежной промышленной AI-разработке.
Как соединяются Скилы и Харнес (День 1), Конвейер ролей и Экспертная панель (День 2), База знаний Карпати и волны GSD (День 3).

ЧАСТЬ 1 // СКИЛЫ, ХАРНЕСЫ И ГАРДРАЙЛЫ

От хаотичного чата к закону репозитория, процедурным контрактам и детерминированным Quality Gates

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

2. Слой 2. Процедурные скилы (SKILL.md)

   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

4 слоя инженерного харнеса

1. Слой 1. Закон репозитория (AGENTS.md)

   Всегда активные правила: границы слоёв и строгие запреты.

2. Слой 2. Процедурные скилы (SKILL.md)

   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

3. Слой 3. Машинные гардрайлы (ruff, basedpyright, pytest)

   Детерминированные ворота качества: нарушение не проходит в мастер.

4 слоя инженерного харнеса

1. **Слой 1. Закон репозитория (AGENTS.md)**
   Всегда активные правила: границы слоёв и строгие запреты.

2. **Слой 2. Процедурные скилы (SKILL.md)**
   Повторяемые процедуры: фазы, критерии готовности, стоп-условия.

3. **Слой 3. Машинные гардрайлы (ruff, basedpyright, pytest)**
   Детерминированные ворота качества: нарушение не проходит в мастер.

4. **Слой 4. Детерминированные Quality Gates**
   Коды возврата 0, characterization tests и независимый ревьюер.

AGENTS.md против SKILL.md: разделение ролей

AGENTS.md: Конституция правил

Роль: неизменяемые законы и границы дозволенного

Фокус: ЧТО ЗАПРЕЩЕНО (не трогать prod-схему, strict typing)

Жизненный цикл: всегда активен, нулевой шаг каждой сессии

Объем: компактный манифест (100-200 строк), навигация и запреты

AGENTS.md против SKILL.md: разделение ролей

AGENTS.md: Конституция правил

Роль: неизменяемые законы и границы дозволенного

Фокус: ЧТО ЗАПРЕЩЕНО (не трогать prod-схему, strict typing)

Жизненный цикл: всегда активен, нулевой шаг каждой сессии

Объем: компактный манифест (100-200 строк), навигация и запреты

SKILL.md: Алгоритмы достижения цели

Роль: пошаговая процедура решения инженерной задачи

Фокус: КАК СДЕЛАТЬ (фазы, чеклисты, стоп-условия, откат)

Жизненный цикл: по требованию (on-demand progressive disclosure)

Объем: рабочий стандарт ≤400 строк (спецификация до 500 строк)

Почему нельзя всё сложить в один большой AGENTS.md?

Ловушка мега-файла (Mega-Prompt Trap)

- Угар контекста: 3000 строк = 30k токенов на каждый шаг
- Constraint Fatigue: модель игнорирует правила в списке
- Lost in the Middle: середина файла вымывается первой
- Информационный шум: чужие правила мешают текущей задаче

Почему нельзя всё сложить в один большой AGENTS.md?

Ловушка мега-файла (Mega-Prompt Trap)

Угар контекста: 3000 строк = 30k токенов на каждый шаг

Constraint Fatigue: модель игнорирует правила в списке

Lost in the Middle: середина файла вымывается первой

Информационный шум: чужие правила мешают текущей задаче

Архитектура Progressive Disclosure

Каталог frontmatter: ~100 токенов в системном индексе

Точечная загрузка: тело скила читается строго под задачу

Экономия контекста: процедуры вынесены из системного промпта

Изоляция знаний: каждый скил автономен и тестируется отдельно

Каркас Agent Skills: файловая структура и уровни L1–L3

Файловая структура каркаса в Git

```
.agents/skills/fastapi-feature/
├── SKILL.md              # L1 + L2
│   ├── frontmatter        # L1 (~100 tok)
│   └── procedural body    # L2 (≤400 строк)
├── scripts/              # L3 Ресурсы
│   └── verify.sh          # Быстрый валидатор
└── references/           # L3 Справочники
    └── contracts.md       # Data-контракты

√ Автономен: 0 внешних облачных зависимостей
```

Каркас Agent Skills: файловая структура и уровни L1–L3

Файловая структура каркаса в Git

```
.agents/skills/fastapi-feature/
├── SKILL.md              # L1 + L2
│   ├── frontmatter        # L1 (~100 tok)
│   └── procedural body    # L2 (≤400 строк)
├── scripts/              # L3 Ресурсы
│   └── verify.sh          # Быстрый валидатор
└── references/           # L3 Справочники
    └── contracts.md       # Data-контракты
```

√ Автономен: 0 внешних облачных зависимостей

Связка уровней: от каталога к исполнению

```
# 1. L1 Frontmatter: системный каталог
#    Имя + описание = ~100 токенов в индексе
# 2. L2 Procedural Body: фазы и DoD
#    Загружается точечно под конкретную задачу
# 3. L3 Supporting Resources: по ссылке
#    Скрипты и схемы читаются строго по нужде
```

√ Чистый контекст: 0 информационного шума
√ Вендор-нейтрально: OpenCode, Cursor, Claude

Анатомия YAML Frontmatter: правила name и description

Поле name: идентификатор процедуры

```
---
name: fastapi-feature
---

# 1. Строгий kebab-case: только латиница и дефис
# 2. Правило 1:1: имя совпадает с
папкой .agents/skills/name/
# 3. Лимит длины: ≤64 символа, глагол + объект
# 4. Уникальность: глобальный ключ в каталоге
репозитория

√ Без пробелов, подчеркиваний и спецсимволов
```

Анатомия YAML Frontmatter: правила name и description

Поле name: идентификатор процедуры

```
---
name: fastapi-feature
---

# 1. Строгий kebab-case: только латиница и дефис
# 2. Правило 1:1: имя совпадает с
папкой .agents/skills/name/
# 3. Лимит длины: ≤64 символа, глагол + объект
# 4. Уникальность: глобальный ключ в каталоге
репозитория

√ Без пробелов, подчеркиваний и спецсимволов
```

Поле description: семантический маршрутизатор

```
description: >-
  Послойно реализует новую фичу FastAPI.
  КОГДА ПРИМЕНЯТЬ: новые эндпоинты, DTO,
сервисы.
  НЕ ПРИМЕНЯТЬ: для багфикса (bugfix),
  рефакторинга (safe-refactoring) или схем БД.
---

# 1. Семантический роутинг: модель выбирает скил
только по нему
# 2. Паттерн двух списков: When to use vs When
NOT to use
# 3. Бюджет: ≤100 токенов (не пересказывать
алгоритм!)
```

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила
Защита от контекстного выгорания	Правила тонут в длинной истории	Всегда активны, занимают контекст каждой сессии	Три уровня: каталог, тело, ресурсы

Сравнение подходов: чат, правила и SKILL.md

Критерий	Голый чат	Правила (Rules)	SKILL.md
Детерминизм процедуры	Реакция случайная, импровизация модели	Нет структуры шагов, зависит от формулировки	Стоп-условия и возврат к фазе скила
Защита от контекстного выгорания	Правила тонут в длинной истории	Всегда активны, занимают контекст каждой сессии	Три уровня: каталог, тело, ресурсы
Переносимость между агентами и IDE	Нулевая: живёт в истории чата	Привязка к одной среде, например Cursor	OpenCode, Cursor, Copilot, Droid, Claude Code

Проектирование скилов: лимиты размера и декомпозиция

Нормы размера (≤400 строк)

Спецификация Agent Skills: индустриальная норма ≤500 строк

Защита от Constraint Fatigue (утомляемости ограничений)

Буфер безопасности (Margin of Safety): рабочий гейт ≤400 строк

Зоны светофора: ≤400 норма, 401–500 техдолг, >500 блокер

Проектирование скилов: лимиты размера и декомпозиция

Нормы размера (≤400 строк)

Спецификация Agent Skills: индустриальная норма ≤500 строк

Защита от Constraint Fatigue (утомляемости ограничений)

Буфер безопасности (Margin of Safety): рабочий гейт ≤400 строк

Зоны светофора: ≤400 норма, 401–500 техдолг, >500 блокер

Декомпозиция монолитов (One Job)

Антипаттерн: мега-скил на 2000 строк ломает контекст

Принцип One Job: одна процедура на одну задачу

Библиотека Pytex: 7 атомарных скилов конвейера

Каноны не копируем, ссылаемся на канонический путь

Детерминированные ворота качества и гардрайлы

pyproject.toml: ruff, basedpyright, pytest-cov

```
[tool.ruff]
line-length = 100

[tool.basedpyright]
typeCheckingMode = "strict"

[tool.pytest.ini_options]
addopts = "-ra -q --strict-markers --cov"

[tool.coverage.report]
fail_under = 85
```

Детерминированные ворота качества и гардрайлы

pyproject.toml: ruff, basedpyright, pytest-cov

```
[tool.ruff]
line-length = 100

[tool.basedpyright]
typeCheckingMode = "strict"

[tool.pytest.ini_options]
addopts = "-ra -q --strict-markers --cov"

[tool.coverage.report]
fail_under = 85
```

Детерминированный верификатор

```
uv run ruff check .
uv run basedpyright
uv run pytest -q
```

Три кода возврата 0 подряд, иначе остановка.

Скил — это не внешняя платформа и не SDK: это исполняемый регламент прямо в Git.

Он живёт в ветке фичи, проходит через Pull Request и исполняется любой моделью.

ЧАСТЬ 2 // ОРКЕСТРАЦИЯ КОНВЕЙЕРА И ПРИЁМКА РАЗНЫМИ

Когнитивный кризис одного контекста, матрица 4 ролей, Structured Handoff и экспертная панель моделей

3 барьера монодиалога: почему один чат не масштабируется

1. Когнитивная слепота

`bias автора`

Модель не видит дефекты в своём коде: она смотрит через призму начальных допущений.

2. Деградация внимания

`Context Rot`

Окно забивается трейсбеками и спорами: системные правила и ограничения вымываются.

3. Иллюзия готовности

`Sycophancy`

Агент бодро рапортует об успехе, пока в продакшене падают зависимые микросервисы.

В одном диалоге сложная задача неизбежно скатывается в бесконечные споры и скрытые баги.

3 барьера монодиалога: почему один чат не масштабируется

1. Когнитивная слепота

`bias автора`

Модель не видит дефекты в своём коде: она смотрит через призму начальных допущений.

2. Деградация внимания

`Context Rot`

Окно забивается трейсбеками и спорами: системные правила и ограничения вымываются.

3. Иллюзия готовности

`Sycophancy`

Агент бодро рапортует об успехе, пока в продакшене падают зависимые микросервисы.

В одном диалоге сложная задача неизбежно скатывается в бесконечные споры и скрытые баги.

3 барьера монодиалога: почему один чат не масштабируется

1. Когнитивная слепота

`bias автора`

Модель не видит дефекты в своём коде: она смотрит через призму начальных допущений.

2. Деградация внимания

`Context Rot`

Окно забивается трейсбеками и спорами: системные правила и ограничения вымываются.

3. Иллюзия готовности

`Sycophancy`

Агент бодро рапортует об успехе, пока в продакшене падают зависимые микросервисы.

В одном диалоге сложная задача неизбежно скатывается в бесконечные споры и скрытые баги.

The Mirror Test Trap: самопроверка в одном контексте

✕ Агент сам себя ревьюит (подгонка)

```
# Агент 'чинит' баг подгонкой теста:
- assert response.status_code == 404
+ assert response.status_code == 200  # чит!

def review_self():
    return 'Код безупречен, тесты зелены'

[!] Слепая зона: тест подогнан под баг.
    Модель не видит дефект своего решения.
```

✅ Независимый Reviewer на чистом diff

```
# Reviewer: 0 токенов контекста + спека:
! ТЕСТОВЫЙ ЧИТ: ослаблен assert в тесте
! Отключенная ссылка обязана отдавать 404.
! Статус: CHANGES_REQUESTED (BLOCKER)

√ Изолированная сессия беспристрастна
√ Аудит строго по specs/task-spec.md и diff
```

The Mirror Test Trap: самопроверка в одном контексте

✕ Агент сам себя ревьюит (подгонка)

```
# Агент 'чинит' баг подгонкой теста:
- assert response.status_code == 404
+ assert response.status_code == 200  # чит!

def review_self():
    return 'Код безупречен, тесты зелены'

[!] Слепая зона: тест подогнан под баг.
    Модель не видит дефект своего решения.
```

✅ Независимый Reviewer на чистом diff

```
# Reviewer: 0 токенов контекста + спека:
! ТЕСТОВЫЙ ЧИТ: ослаблен assert в тесте
! Отключенная ссылка обязана отдавать 404.
! Статус: CHANGES_REQUESTED (BLOCKER)

√ Изолированная сессия беспристрастна
√ Аудит строго по specs/task-spec.md и diff
```

Эффект поддакивания (Sycophancy) и безопасность

✕ Утечка авторизации (CWE-532)

```
# Автор: 'Выведи токен в лог для отладки'
# Агент соглашается с опасной просьбой:
+ auth =
request.headers.get('authorization')
+ logger.info(f'Token: {auth}')  # УТЕЧКА!

[!] CWE-532: Секретный Bearer-токен
    утёк в stdout, мониторинг и логи.
```

✅ Защита гардрайлом Zero Auth Leak

```
# Гардрайл блокирует утечки в PR:
- logger.info(f'Token: {auth}')
+ logger.info('Request authenticated')

√ Запрет логирования Authorization/Cookie
√ Проверка regex на токены в CI-гейте
√ Блокирующий статус CHANGES_REQUESTED
```

Эффект поддакивания (Sycophancy) и безопасность

✕ Утечка авторизации (CWE-532)

```
# Автор: 'Выведи токен в лог для отладки'
# Агент соглашается с опасной просьбой:
+ auth =
request.headers.get('authorization')
+ logger.info(f'Token: {auth}')  # УТЕЧКА!

[!] CWE-532: Секретный Bearer-токен
    утёк в stdout, мониторинг и логи.
```

✅ Защита гардрайлом Zero Auth Leak

```
# Гардрайл блокирует утечки в PR:
- logger.info(f'Token: {auth}')
+ logger.info('Request authenticated')

√ Запрет логирования Authorization/Cookie
√ Проверка regex на токены в CI-гейте
√ Блокирующий статус CHANGES_REQUESTED
```

Context Rot: деградация внимания в длинном контексте

app/api/v1/links.py (эрозия архитектурных правил)

```
# Начало диалога (токен 500): модель строго соблюдает AGENTS.md
async def get_link(db: AsyncSession = Depends(get_db)): ...

# ... 40 сообщений спустя (токен 45 000, трейсбеки, черновики) ...
# Модель забыла инвариант: прямое открытие соединения мимо Depends!
def toggle_link_status(link_id: str):
    conn = sqlite3.connect("app.db")  # ОБХОД DI И ПУЛА СОЕДИНЕНИЙ
    conn.execute(f"UPDATE links SET status='{status}'")  # SQL-ИНЪЕКЦИЯ
    conn.commit()
```

Монодиалог в чате vs Изолированные сессии конвейера

Критерий	Монодиалог в одном чате	Изолированные сессии конвейера
Объём контекста	Разрастается до 50k+ токенов, внимание деградирует	Каждая роль стартует с 0 токенов и сжатого пакета
Беспристрастность	Автор защищает свой код и поддакивает пользователю	Ревьюер критикует diff с чистого листа
Права доступа	Полный доступ на запись во все файлы проекта	Строгое разделение: Coder (app/), Reviewer (Read-Only)
Передача состояния	Неструктурированная переписка и трейсбеки	Спецификация SDD и сжатый handoff-пакет (1–3 КБ)

Матрица конвейера: 4 роли и разграничение прав

Роль 1

Coder

`WRITE: app/, tests/`

Реализует фичу по спеке, пишет unit/integration тесты под защитой TDD

Роль 2

Reviewer

`STRICT READ-ONLY`

Аудирует git diff, чистоту слоёв и соответствие спеке в свежем контексте

Роль 3

Summarizer

`STRICT READ-ONLY`

Формирует доказательный PR description, сверяет claim ledger и handoff

Роль 4

Techlead Gate

`STRICT READ-ONLY`

5 критериев приёмки, пакет доказательств, право безусловного вето

Инвариант: Ни одна роль не объединяет разработку, независимое ревью и право выпуска в релиз.

Разделение прав: один исполнитель vs Конвейер

✕ Совмещение ролей (Вайб-кодинг)

```
# Полный доступ ко всем файлам репо:
WRITE: app/main.py       (бизнес-логика)
WRITE: tests/test_*.py   (подгонка тестов)
WRITE: alembic/versions  (опасная схема)

[!] Агент сам пишет и сам меняет тесты.
    Нулевой контроль архитектурных границ.
```

✅ Изолированные сессии и права

```
# Разделение привилегий по ролям:
Coder:      WRITE в app/ & tests/ (спека)
Reviewer:   STRICT READ-ONLY (аудит diff)
Summarizer: STRICT READ-ONLY (PR & ledger)
Gate:       STRICT READ-ONLY (право вето)

√ Разделение кода, тестов и приёмки
```

Разделение прав: один исполнитель vs Конвейер

✕ Совмещение ролей (Вайб-кодинг)

```
# Полный доступ ко всем файлам репо:
WRITE: app/main.py      (бизнес-логика)
WRITE: tests/test_*.py  (подгонка тестов)
WRITE: alembic/versions (опасная схема)

[!] Агент сам пишет и сам меняет тесты.
    Нулевой контроль архитектурных границ.
```

✅ Изолированные сессии и права

```
# Разделение привилегий по ролям:
Coder:      WRITE в app/ & tests/ (спека)
Reviewer:   STRICT READ-ONLY (аудит diff)
Summarizer: STRICT READ-ONLY (PR & ledger)
Gate:       STRICT READ-ONLY (право вето)

√ Разделение кода, тестов и приёмки
```

Протокол передачи работы: Чат vs Structured Handoff

Передача через историю диалога

- Разрастание контекста до мегабайтов текста
- Потеря ключевых требований в массиве сообщений
- Высокая стоимость токенов на каждом запросе
- Шумные трейсбеки и черновики в памяти модели

Машинный пакет Structured Handoff

- Компактный файл на диске объёмом 1–3 КБ
- Чёткая структура: Артефакты, Решения, Риски
- Свежий контекст для следующего исполнителя
- Воспроизводимость и аудит всей цепочки правок

Протокол передачи работы: Чат vs Structured Handoff

Передача через историю диалога

- Разрастание контекста до мегабайтов текста
- Потеря ключевых требований в массиве сообщений
- Высокая стоимость токенов на каждом запросе
- Шумные трейсбеки и черновики в памяти модели

Машинный пакет Structured Handoff

- Компактный файл на диске объёмом 1–3 КБ
- Чёткая структура: Артефакты, Решения, Риски
- Свежий контекст для следующего исполнителя
- Воспроизводимость и аудит всей цепочки правок

Анатомия карточки handoff-coder.md

1. Ссылки на артефакты

`git diff & commits`

Точные пути к изменённым файлам, хеши коммитов и затронутые эндпоинты.

2. Принятые решения

`Design Decisions`

Обоснование выбора типов, структуры индексов и алгоритмов валидации.

3. Риски и допущения

`Residual Risks`

Явное перечисление того, что не вошло в скоуп или требует проверки под нагрузкой.

Карточка handoff-coder.md создаётся Кодером на диске и передаётся Ревьюеру.

Circuit Breaker: лимит двух кругов итераций

47 Круг 1: Кодер реализует фичу -> Ревьюер выдаёт замечания по diff

47 Круг 2: Кодер устраняет только названные замечания -> Ревьюер проверяет дельту

47 Срабатывание Circuit Breaker: если дефекты не устранены за 2 круга — немедленный стоп

47 Эскалация: задача передаётся архитектору или техлиду для пересмотра спеки

Red-Green TDD: разработка под защитой падающих тестов

Шаг 1. Спека

`specs/task-spec.md`

Фиксация контракта и граничных условий задачи (SDD)

Шаг 2. Red-тест

`test_link_toggle.py`

Кодер пишет падающий тест ДО реализации (Exit != 0)

Шаг 3. Green-код

`app/services/link.py`

Кодер реализует минимум логики до зелёных тестов (Exit 0)

Шаг 4. Мутация

`Mutation Drill`

Инвертируем флаг: тест обязан поймать баг и упасть (Red)

Инвариант: Тест, который не был красным до написания кода, не может служить доказательством работоспособности.

Проверка качества тестов: Иллюзия vs Мутация

✕ Поверхностный тест (Vibe-тест)

```
async def test_toggle_link_vibe(client):
    res = await client.patch('/toggle')
    assert res.status_code == 200
    # Запись в БД и флаг не проверены!

[!] Ложная зелень: пустой 200 OK,
    а ссылка в БД не переключилась.
```

✅ Мутационный дрилл (Mutation Drill)

```
# Мутируем app/main.py (дефект):
# link.disabled = False  (всегда активна!)
>   assert link.disabled is True
E   AssertionError: assert False is True
E   FAILED tests/test_toggle.py - exit 1

√ Тест обязан упасть при внесении бага
√ Доказана чувствительность к дефектам
```

Проверка качества тестов: Иллюзия vs Мутация

✕ Поверхностный тест (Vibe-тест)

```
async def test_toggle_link_vibe(client):
    res = await client.patch('/toggle')
    assert res.status_code == 200
    # Запись в БД и флаг не проверены!

[!] Ложная зелень: пустой 200 OK,
    а ссылка в БД не переключилась.
```

✅ Мутационный дрилл (Mutation Drill)

```
# Мутируем app/main.py (дефект):
# link.disabled = False  (всегда активна!)
>   assert link.disabled is True
E   AssertionError: assert False is True
E   FAILED tests/test_toggle.py - exit 1

√ Тест обязан упасть при внесении бага
√ Доказана чувствительность к дефектам
```

5 критериев приёмки техлида (Techlead Gate)

1. Спецификация

`SDD Match`

100% соответствие контракту specs/task-spec.md.

2. Инварианты

`Layers`

Соблюдение границ слоёв Router -> Service -> Repo.

3. Quality Gates

`Exit 0`

Автоматический арбитр истины: ruff, basedpyright, pytest.

4. Zero Auth Leak

`CWE-532`

Гарантированное отсутствие токенов в логах.

5. Blast Radius

`Scope`

Дифф строго в рамках разрешённых 4 файлов.

Все 5 критериев приёмки должны быть подтверждены объективными артефактами на диске.

Приёмка релиза: Неформальный аппрув vs Пакет доказательств

✕ Неформальный аппрув в чате

```
# Разработчик: 'Ну что, фича готова?'
# Агент: 'Да, всё проверил, можно
мержить!'

[!] Нулевая доказательная база:
    - Нет фиксации Blast Radius
    - Не проверена обратимость downgrade()
    - Нет логов мутационного дрилла
    - Субъективная оценка вместо фактов
```

✅ Машинный пакет proof-package.json

```
{
  "ticket": "TASK-42",
  "blast_radius_files": 4,
  "quality_gates": {"ruff": 0, "pytest":
15},
  "mutation_drill_passed": true,
  "cwe_532_clean": true,
  "verdict": "ACCEPT"
}
√ 5 объективных критериев приёмки
```

Приёмка релиза: Неформальный аппрув vs Пакет доказательств

✕ Неформальный аппрув в чате

```
# Разработчик: 'Ну что, фича готова?'
# Агент: 'Да, всё проверил, можно
мержить!'

[!] Нулевая доказательная база:
    - Нет фиксации Blast Radius
    - Не проверена обратимость downgrade()
    - Нет логов мутационного дрилла
    - Субъективная оценка вместо фактов
```

✅ Машинный пакет proof-package.json

```
{
  "ticket": "TASK-42",
  "blast_radius_files": 4,
  "quality_gates": {"ruff": 0, "pytest":
15},
  "mutation_drill_passed": true,
  "cwe_532_clean": true,
  "verdict": "ACCEPT"
}
√ 5 объективных критериев приёмки
```

Приёмка разными моделями: Совет Тимлидов и кворум 2/3

Принцип независимого арбитража

• 3 независимых модели разных семейств (DeepSeek, GLM, Kimi / Claude / GPT)
• Слепое голосование: судьи не видят решений друг друга (отсечение хор-эффекта)
• Пакет доказательств: судьи читают артефакты Proof Package на диске, а не чат
• Никаких обещаний: слова модели «всё проверил» не имеют силы для судьи

Кворум 2/3 и абсолютное вето

• Вердикт READY требует консенсуса большинства не менее 2/3 голосов
• Абсолютное вето: дефект безопасности (CWE-532, утечка) не снимается большинством
• Закрытые дебаты: при расхождении мнений проводится раунд анонимных аргументов
• Защита от слепых зон: исключает субъективную предвзятость одиночной модели

Детерминированная приёмка: техлид судит по объективным файлам доказательств на диске, а не по обещаниям модели.

Если хотя бы один критерий приёмки не закрыт файлом — слияние в main запрещено.

ЧАСТЬ 1 // БАЗА ЗНАНИЙ КАРПАТИ

ЧАСТЬ 3 // БАЗА ЗНАНИЙ КАРПАТИ И МЕТОДОЛОГИЯ GSD

Масштабирование на большие проекты: физика внимания, LLM Wiki и конечный автомат на диске

Физика внимания: почему нельзя скормить модели весь монолит

16,4 млрд

СВЯЗЕЙ ВНИМАНИЯ НА 128K ТОКЕНОВ

При росте окна с 10k до 128k токенов число парных связей Self-Attention растёт квадратично: в 164 раза.

15-35%

УДЕРЖАНИЕ ПРАВИЛ В СЕРЕДИНЕ ОКНА

Эффект Lost in the Middle: инструкции в зоне 35-70% раздутого окна почти перестают влиять на генерацию кода.

≤ 15 000

ОПТИМУМ РАБОЧЕГО НАБОРА В RAM

Максимальный объём активных токенов, при котором агент удерживает до 98% перекрёстных архитектурных правил.

Суть метода Карпати: База знаний как компилятор (LLM Wiki)

«Obsidian - это IDE, LLM - программист, а вики - компилируемая кодовая база знаний»

01 / RAW (ИСХОДНИКИ)

Папка `raw/inbox/`, исходный код `app/`, миграции, внешние спецификации и постановочные тикеты.

Это неизменяемые первоисточники фактов: агент опирается на них, но не переписывает сырые входы задним числом.

В классическом чате агент каждый раз пытается заново «переварить» сырые исходники с нуля.

02 / WIKI (БИНАРЬ ЗНАНИЙ)

Двухуровневое дерево: Уровень 1 docs/INDEX.md (<=100 строк, ~1.5k токенов) -> Уровень 2 docs/domains/*.md (50-120 строк).

Территория агента: разработчик читает, а компилирует, перелинковывает и обновляет страницы ИИ-агент.
Анатомия карточки: 5 обязательных разделов с якорем Location (точные пути к коду на диске) и нерушимыми правилами.

03 / ПРАВИЛА И ВАЛИДАЦИЯ

Физическое разделение: в docs/ только чистый Markdown! Скрипты и тесты живут строго в tools/.
Три операции: Ingest (впитывание по Filing Back), Query (подкачка в RAM), Lint (контроль сходимости).
Автоматический валидатор tools/verify_kb_sync.py сверяет пути и размер, возвращая exit 1 при любом дрейфе.

Суть метода Карпати: База знаний как компилятор (LLM Wiki)

«Obsidian - это IDE, LLM - программист, а вики - компилируемая кодовая база знаний»

01 / RAW (ИСХОДНИКИ)

Папка `raw/inbox/`, исходный код `app/`, миграции, внешние спецификации и постановочные тикеты.

Это неизменяемые первоисточники фактов: агент опирается на них, но не переписывает сырые входы задним числом.

В классическом чате агент каждый раз пытается заново «переварить» сырые исходники с нуля.

02 / WIKI (БИНАРЬ ЗНАНИЙ)

Двухуровневое дерево: Уровень 1 docs/INDEX.md (<=100 строк, ~1.5k токенов) -> Уровень 2 docs/domains/*.md (50-120 строк).

Территория агента: разработчик читает, а компилирует, перелинковывает и обновляет страницы ИИ-агент.
Анатомия карточки: 5 обязательных разделов с якорем Location (точные пути к коду на диске) и нерушимыми правилами.

03 / ПРАВИЛА И ВАЛИДАЦИЯ

Физическое разделение: в docs/ только чистый Markdown! Скрипты и тесты живут строго в tools/.
Три операции: Ingest (впитывание по Filing Back), Query (подкачка в RAM), Lint (контроль сходимости).
Автоматический валидатор tools/verify_kb_sync.py сверяет пути и размер, возвращая exit 1 при любом дрейфе.

Суть метода Карпати: База знаний как компилятор (LLM Wiki)

«Obsidian - это IDE, LLM - программист, а вики - компилируемая кодовая база знаний»

01 / RAW (ИСХОДНИКИ)

Папка `raw/inbox/`, исходный код `app/`, миграции, внешние спецификации и постановочные тикеты.

Это неизменяемые первоисточники фактов: агент опирается на них, но не переписывает сырые входы задним числом.

В классическом чате агент каждый раз пытается заново «переварить» сырые исходники с нуля.

02 / WIKI (БИНАРЬ ЗНАНИЙ)

Двухуровневое дерево: Уровень 1 docs/INDEX.md (<=100 строк, ~1.5k токенов) -> Уровень 2 docs/domains/*.md (50-120 строк).

Территория агента: разработчик читает, а компилирует, перелинковывает и обновляет страницы ИИ-агент.
Анатомия карточки: 5 обязательных разделов с якорем Location (точные пути к коду на диске) и нерушимыми правилами.

03 / ПРАВИЛА И ВАЛИДАЦИЯ

Физическое разделение: в docs/ только чистый Markdown! Скрипты и тесты живут строго в tools/.
Три операции: Ingest (впитывание по Filing Back), Query (подкачка в RAM), Lint (контроль сходимости).
Автоматический валидатор tools/verify_kb_sync.py сверяет пути и размер, возвращая exit 1 при любом дрейфе.

Каркас базы знаний проекта в репозитории

```
fastapi/
├── docs/                  # БАЗА ЗНАНИЙ (чистый Markdown, без скриптов!)
│   ├── INDEX.md           # Уровень 1: роутер проекта (⩽ 100 строк, ~1.5k токенов)
│   └── domains/           # Уровень 2: карточки доменов (по 50-120 строк каждая)
│       ├── core.md        # Домен Core: протоколы ядра, SSRF, настройки
│       ├── links.md       # Домен Links: редиректы 307, короткие ссылки
│       ├── analytics.md   # Домен Analytics: клики, агрегаты статистики
│       └── billing.md     # Домен Billing: списания, тарифы, баланс в копейках
├── tools/                 # СКРИПТЫ ВАЛИДАЦИИ (отделены от базы знаний!)
│   └── verify_kb_sync.py  # Автоматический валидатор сходимости базы с кодом
└── app/                  # ИСХОДНЫЙ КОД (сотни файлов на диске, не в окне!)
    └── schemas/, core/   # Агент подгружает в RAM только 2-4 файла из Location
```

Физическое разделение: в docs/ только чистый Markdown (никаких скриптов!), валидаторы строго в tools/. Агент читает INDEX.md -> одну карточку -> 2-4 файла кода из раздела Location.

Пример каркаса (1/2): Корневой навигатор docs/INDEX.md

```
# Индекс доменов проекта fastapi-linksvc (v2.4.0)
> Слои: schemas → core → services → api (обратные импорты запрещены)
> Бюджет задачи: ⩽ 7 файлов, ⩽ 400 строк добавления

| Домен     | Код (app/)                    | Карточка     |
|-----------|-------------------------------|--------------|
| Core      | core/{config,protocols}.py    | core.md      |
| Links     | {schemas,services}/links.py   | links.md     |
| Analytics | services/analytics.py         | analytics.md |
| Billing   | {schemas,services}/billing    | billing.md   |

## Ключевые архитектурные правила
1. Роуты app/api/ принимают и отдают только Pydantic DTO.
2. Сервисы ходят в БД только через app/core/protocols.py.
```

Шапка архитектурных границ: жесткие слои schemas -> core -> services -> api и бюджет задачи не более 7 файлов.

Пример каркаса (1/2): Корневой навигатор docs/INDEX.md

```
# Индекс доменов проекта fastapi-linksvc (v2.4.0)
> Слои: schemas → core → services → api (обратные импорты запрещены)
> Бюджет задачи: ⩽ 7 файлов, ⩽ 400 строк добавления

| Домен     | Код (app/)                    | Карточка     |
|-----------|-------------------------------|--------------|
| Core      | core/{config,protocols}.py    | core.md      |
| Links     | {schemas,services}/links.py   | links.md     |
| Analytics | services/analytics.py         | analytics.md |
| Billing   | {schemas,services}/billing    | billing.md   |

## Ключевые архитектурные правила
1. Роуты app/api/ принимают и отдают только Pydantic DTO.
2. Сервисы ходят в БД только через app/core/protocols.py.
```

Таблица маршрутизации: имя домена, ключевые пути кода и относительная ссылка на карточку docs/domains/<домен>.md.

Пример каркаса (1/2): Корневой навигатор docs/INDEX.md

```
# Индекс доменов проекта fastapi-linksvc (v2.4.0)
> Слои: schemas → core → services → api (обратные импорты запрещены)
> Бюджет задачи: ⩽ 7 файлов, ⩽ 400 строк добавления

| Домен     | Код (app/)                    | Карточка     |
|-----------|-------------------------------|--------------|
| Core      | core/{config,protocols}.py    | core.md      |
| Links     | {schemas,services}/links.py   | links.md     |
| Analytics | services/analytics.py         | analytics.md |
| Billing   | {schemas,services}/billing    | billing.md   |

## Ключевые архитектурные правила
1. Роуты app/api/ принимают и отдают только Pydantic DTO.
2. Сервисы ходят в БД только через app/core/protocols.py.
```

Архитектурные правила в конце индекса: размещены в зоне 90-99% внимания модели, где правила удерживаются надежнее всего.

Как агент работает с базой знаний: постраничная подкачка в RAM

Шаг навигации	Что загружается в окно (RAM)	Объём в окне	Что остаётся на диске
Шаг 1. Ориентирование	Только роутер `docs/INDEX.md`	~1 500 токенов	Весь исходный код app/ и спецификации
Шаг 2. Выбор домена	Индекс + 1 карточка `docs/domains/billing.md`	~3 000 токенов	Карточки core, links, analytics на диске
Шаг 3. Загрузка кода	Только 2-4 файла Location (Data Contract DTO + логика)	~12 000 токенов	Остальные 40+ файлов проекта на диске
Шаг 4. Filing Back	Расширение схемы + подшивка в карточку + `verify_kb_sync`	До ~12 000 токенов	92-95% экономии RAM (vs 150k+ без базы знаний)

Как агент работает с базой знаний: постраничная подкачка в RAM

Шаг навигации	Что загружается в окно (RAM)	Объём в окне	Что остаётся на диске
Шаг 1. Ориентирование	Только роутер `docs/INDEX.md`	~1 500 токенов	Весь исходный код app/ и спецификации
Шаг 2. Выбор домена	Индекс + 1 карточка `docs/domains/billing.md`	~3 000 токенов	Карточки core, links, analytics на диске
Шаг 3. Загрузка кода	Только 2-4 файла Location (Data Contract DTO + логика)	~12 000 токенов	Остальные 40+ файлов проекта на диске
Шаг 4. Filing Back	Расширение схемы + подшивка в карточку + `verify_kb_sync`	До ~12 000 токенов	92-95% экономии RAM (vs 150k+ без базы знаний)

Как агент работает с базой знаний: постраничная подкачка в RAM

Шаг навигации	Что загружается в окно (RAM)	Объём в окне	Что остаётся на диске
Шаг 1. Ориентирование	Только роутер `docs/INDEX.md`	~1 500 токенов	Весь исходный код app/ и спецификации
Шаг 2. Выбор домена	Индекс + 1 карточка `docs/domains/billing.md`	~3 000 токенов	Карточки core, links, analytics на диске
Шаг 3. Загрузка кода	Только 2-4 файла Location (Data Contract DTO + логика)	~12 000 токенов	Остальные 40+ файлов проекта на диске
Шаг 4. Filing Back	Расширение схемы + подшивка в карточку + `verify_kb_sync`	До ~12 000 токенов	92-95% экономии RAM (vs 150k+ без базы знаний)

Как агент работает с базой знаний: постраничная подкачка в RAM

Шаг навигации	Что загружается в окно (RAM)	Объём в окне	Что остаётся на диске
Шаг 1. Ориентирование	Только роутер `docs/INDEX.md`	~1 500 токенов	Весь исходный код app/ и спецификации
Шаг 2. Выбор домена	Индекс + 1 карточка `docs/domains/billing.md`	~3 000 токенов	Карточки core, links, analytics на диске
Шаг 3. Загрузка кода	Только 2-4 файла Location (Data Contract DTO + логика)	~12 000 токенов	Остальные 40+ файлов проекта на диске
Шаг 4. Filing Back	Расширение схемы + подшивка в карточку + `verify_kb_sync`	До ~12 000 токенов	92-95% экономии RAM (vs 150k+ без базы знаний)

Регулярная гигиена базы знаний: 7 правил аудита kb-lint

Правило проверки	Что ищет агент-аудитор в базе знаний	Действие при обнаружении
1. Противоречия	Разные цифры, лимиты или правила в двух файлах	Сверить с кодом, пометить актуальное с датой
2. Битые ссылки и сироты	Путь из INDEX.md или Location удалён; карточка не в индексе	Починить путь или подключить страницу в `INDEX.md`
3. Дубли (2 источника)	Один и тот же контракт переписан в 2-3 местах	Оставить канон в одной карточке, дать ссылку
4. Пробелы (Gaps)	Модуль есть в app/, но не описан ни в одной карточке	Дописать в Location или создать новую карточку
5. Факты без даты	Слова «сейчас», «временно», «скоро» без YYYY-MM-DD	Проставить точную дату и статус актуальности
6. Раздутые файлы	Индекс > 100 строк или карточка домена > 100 строк	Вынести детали в дочернюю страницу, сжать индекс
7. Дрейф сигнатур	Сигнатура метода в коде изменилась, карточка устарела	Обновить карточку (Filing Back), `verify_kb_sync.py`

Суть проблемы: 4 фазы распада длинной сессии (Context Rot)

Фаза сессии	Шаги и окно	Что происходит в контексте	Результат в коде
1. Высокая точность	Шаги 1-4 (0-15k)	Чистое окно, правила и карточка в фокусе	95% точность, чистые типы и контракты
2. Трейсбек-шум	Шаги 5-8 (15-45k)	В чат вставлены 120 строк логов `pytest`	Внимание уходит на фреймворк Starlette
3. Амнезия и дрейф	Шаги 9-14 (45-90k)	Правила попали в мёртвую зону 35-70% окна	Прямой SQL в роутере, захват чужих файлов
4. Коллапс сессии	Шаги 15-20 (90-198k)	92% окна: старые ошибки и извинения	Циклический импорт, `git reset --hard`

Суть GSD: Три опоры защиты от деградации контекста

Вместо одного бесконечного чата: конечный автомат на диске и свежие окна субагентов

01 / ПАМЯТЬ В .PLANNING/

Вся память о цели, требованиях, фазах и текущей позиции хранится на диске в папке `.planning/`.

Новая сессия стартует с чтения файлов состояния за 2 секунды, а не с пересказа длинной переписки.

Агент не решает сам, на каком он этапе: фаза жёстко записана в `STATE.md`.

02 / ПЛАН - ЭТО ПРОМПТ

Каждая фаза режется на планы по 2-3 атомарные задачи с бюджетом до 50% контекстного окна.

Файл `01-01-PLAN.md` не пересказывают модели: он сам является готовым промптом исполнения.

Внутри плана для каждой задачи заданы точные файлы, действие, команда проверки и критерий `done`.

03 / ВОЛНЫ СУБАГЕНТОВ

Каждая волна (`wave: 1`, `wave: 2`) идёт в новом чистом окне (~200k токенов, 0% мусора).

Независимые задачи волны идут параллельно по непересекающимся файлам; зависимые: в следующей волне.

Каждая задача завершается атомарным коммитом и запиской `Structured Handoff` (до 15 строк).

Суть GSD: Три опоры защиты от деградации контекста

Вместо одного бесконечного чата: конечный автомат на диске и свежие окна субагентов

01 / ПАМЯТЬ В .PLANNING/

Вся память о цели, требованиях, фазах и текущей позиции хранится на диске в папке `.planning/`.

Новая сессия стартует с чтения файлов состояния за 2 секунды, а не с пересказа длинной переписки.

Агент не решает сам, на каком он этапе: фаза жёстко записана в `STATE.md`.

02 / ПЛАН - ЭТО ПРОМПТ

Каждая фаза режется на планы по 2-3 атомарные задачи с бюджетом до 50% контекстного окна.

Файл `01-01-PLAN.md` не пересказывают модели: он сам является готовым промптом исполнения.

Внутри плана для каждой задачи заданы точные файлы, действие, команда проверки и критерий `done`.

03 / ВОЛНЫ СУБАГЕНТОВ

Каждая волна (`wave: 1`, `wave: 2`) идёт в новом чистом окне (~200k токенов, 0% мусора).

Независимые задачи волны идут параллельно по непересекающимся файлам; зависимые: в следующей волне.

Каждая задача завершается атомарным коммитом и запиской `Structured Handoff` (до 15 строк).

Суть GSD: Три опоры защиты от деградации контекста

Вместо одного бесконечного чата: конечный автомат на диске и свежие окна субагентов

01 / ПАМЯТЬ В .PLANNING/

Вся память о цели, требованиях, фазах и текущей позиции хранится на диске в папке `.planning/`.

Новая сессия стартует с чтения файлов состояния за 2 секунды, а не с пересказа длинной переписки.

Агент не решает сам, на каком он этапе: фаза жёстко записана в `STATE.md`.

02 / ПЛАН - ЭТО ПРОМПТ

Каждая фаза режется на планы по 2-3 атомарные задачи с бюджетом до 50% контекстного окна.

Файл `01-01-PLAN.md` не пересказывают модели: он сам является готовым промптом исполнения.

Внутри плана для каждой задачи заданы точные файлы, действие, команда проверки и критерий `done`.

03 / ВОЛНЫ СУБАГЕНТОВ

Каждая волна (`wave: 1`, `wave: 2`) идёт в новом чистом окне (~200k токенов, 0% мусора).

Независимые задачи волны идут параллельно по непересекающимся файлам; зависимые: в следующей волне.

Каждая задача завершается атомарным коммитом и запиской `Structured Handoff` (до 15 строк).

Каркас GSD: анатомия папки .planning/ (единый источник правды)

```
.planning/
├── PROJECT.md            # Вижен, стек, глобальные ограничения
├── REQUIREMENTS.md       # Требования v1 / v2 / вне охвата
├── ROADMAP.md            # Фазы и волны, статусы [x]
├── STATE.md              # Активная фаза, Retry Count, Handoff
├── codebase/             # Карта существующего кода
└── phases/01-billing/    # Артефакты одной фазы разработки
    ├── 01-CONTEXT.md         # Принятые развилки API и БД
    ├── 01-RESEARCH.md        # Находки по коду до плана
    ├── 01-01-PLAN.md         # План-промпт, 2-3 задачи
    ├── 01-01-SUMMARY.md      # Итог выполнения и хеши коммитов
    └── 01-VERIFICATION.md    # Чек-лист приёмки фазы
```

Четыре корневых файла управления: PROJECT.md, REQUIREMENTS.md, ROADMAP.md, STATE.md.

Каркас GSD: анатомия папки .planning/ (единый источник правды)

```
.planning/
├── PROJECT.md            # Вижен, стек, глобальные ограничения
├── REQUIREMENTS.md       # Требования v1 / v2 / вне охвата
├── ROADMAP.md            # Фазы и волны, статусы [x]
├── STATE.md              # Активная фаза, Retry Count, Handoff
├── codebase/             # Карта существующего кода
└── phases/01-billing/    # Артефакты одной фазы разработки
    ├── 01-CONTEXT.md         # Принятые развилки API и БД
    ├── 01-RESEARCH.md        # Находки по коду до плана
    ├── 01-01-PLAN.md         # План-промпт, 2-3 задачи
    ├── 01-01-SUMMARY.md      # Итог выполнения и хеши коммитов
    └── 01-VERIFICATION.md    # Чек-лист приёмки фазы
```

Папка `.planning/codebase/` создаётся перед стартом в существующем репозитории (Brownfield).

Каркас GSD: анатомия папки .planning/ (единый источник правды)

```
.planning/
├── PROJECT.md            # Вижен, стек, глобальные ограничения
├── REQUIREMENTS.md       # Требования v1 / v2 / вне охвата
├── ROADMAP.md            # Фазы и волны, статусы [x]
├── STATE.md              # Активная фаза, Retry Count, Handoff
├── codebase/             # Карта существующего кода
└── phases/01-billing/    # Артефакты одной фазы разработки
    ├── 01-CONTEXT.md         # Принятые развилки API и БД
    ├── 01-RESEARCH.md        # Находки по коду до плана
    ├── 01-01-PLAN.md         # План-промпт, 2-3 задачи
    ├── 01-01-SUMMARY.md      # Итог выполнения и хеши коммитов
    └── 01-VERIFICATION.md    # Чек-лист приёмки фазы
```

Пять файлов жизненного цикла одной фазы: CONTEXT -> RESEARCH -> PLAN -> SUMMARY -> VERIFICATION.

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

77 ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ `/gsd-map-codebase → /gsd-new-project`

77 ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА `/gsd-discuss-phase N`

77 ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА `/gsd-plan-phase N + plan_checker`

77 ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ `/gsd-execute-phase N`

77 ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ `/gsd-verify-work N`

77 ШАГ 6. ЭСТАФЕТА И ВЫПУСК `/gsd-ship`

Для существующего монолита строим карту `.planning/codebase/`, затем формируем `PROJECT.md`, `REQUIREMENTS.md` (с проверками) и `ROADMAP.md`.

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

78 ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ `/gsd-map-codebase` → `/gsd-new-project`

78 ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА `/gsd-discuss-phase N`

78 ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА `/gsd-plan-phase N + plan_checker`

78 ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ `/gsd-execute-phase N`

78 ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ `/gsd-verify-work N`

78 ШАГ 6. ЭСТАФЕТА И ВЫПУСК `/gsd-ship`

Выявляем серые зоны (тип баланса `int` в копейках, коды ошибок 402/503, идемпотентность) и записываем решения в `01-CONTEXT.md`.

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

```
79  ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ  /gsd-map-codebase → /gsd-new-project
79  ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА  /gsd-discuss-phase N
79  ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА  /gsd-plan-phase N + plan_checker
79  ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ  /gsd-execute-phase N
79  ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ  /gsd-verify-work N
79  ШАГ 6. ЭСТАФЕТА И ВЫПУСК  /gsd-ship
```

Строим атомарные `01-XX-PLAN.md` (по 2-3 задачи) и прогоняем `plan_checker.py` (до 3 итераций сверки с `REQUIREMENTS.md` и бюджетом файлов).

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

80 ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ `/gsd-map-codebase` → `/gsd-new-project`

80 ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА `/gsd-discuss-phase N`

80 ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА `/gsd-plan-phase N + plan_checker`

80 ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ `/gsd-execute-phase N`

80 ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ `/gsd-verify-work N`

80 ШАГ 6. ЭСТАФЕТА И ВЫПУСК `/gsd-ship`

Субагенты выполняют волны (`wave: 1`, `wave: 2`) в свежих окнах по 200k токенов; каждая задача фиксируется отдельным коммитом.

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

81 ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ `/gsd-map-codebase` → `/gsd-new-project`

81 ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА `/gsd-discuss-phase N`

81 ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА `/gsd-plan-phase N + plan_checker`

81 ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ `/gsd-execute-phase N`

81 ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ `/gsd-verify-work N`

81 ШАГ 6. ЭСТАФЕТА И ВЫПУСК `/gsd-ship`

Скрипт и чек-лист `01-VERIFICATION.md` проверяют реальные сценарии, изоляцию слоёв и бюджет зоны влияния.

Какие шаги в GSD: полный цикл от разведки до сдачи фичи

```
82  ШАГ 1. РАЗВЕДКА И ИНИЦИАЛИЗАЦИЯ  /gsd-map-codebase → /gsd-new-project
82  ШАГ 2. ФИКСАЦИЯ РАЗВИЛОК ДО ПЛАНА  /gsd-discuss-phase N
82  ШАГ 3. ПЛАНИРОВАНИЕ И ПРОВЕРКА ДРЕЙФА  /gsd-plan-phase N + plan_checker
82  ШАГ 4. ИСПОЛНЕНИЕ ВОЛНАМИ  /gsd-execute-phase N
82  ШАГ 5. ПРИЁМКА И ГРАНИЧНЫЕ СЛУЧАИ  /gsd-verify-work N
82  ШАГ 6. ЭСТАФЕТА И ВЫПУСК  /gsd-ship
```

В `STATE.md` пишется сжатый `Structured Handoff` (до 15 строк), в `ROADMAP.md` ставится `[x]`, формируется готовый PR.

Фаза 3: Скоринг пузырьковым методом и передача эстафеты (STATE.md)

```
# Фаза 3: Скоринг и ранжирование (Финальная сессия)
- Сессия 3 стартует в чистом окне, читает .planning/STATE.md и data/parsed.jsonl.
- Алгоритм скоринга: пузырьковым методом сравнивает пары записей по метрикам
  (релевантность, полнота, свежесть) и всплывает лучший контент наверх.
- Сохраняет отсортированный результат в report.csv.
- Записывает финал в .planning/STATE.md: «Ранжирование завершено, exit 0».


── Почему наивный чат здесь умирает, а GSD побеждает ──
В едином чате: к моменту скоринга окно забито логами краулера и сырым HTML.
Модель ловит Lost in the Middle, путает поля и ломает сортировку.
В конвейере GSD: третья сессия имеет 100% внимания на алгоритме скоринга.
Память процесса живёт не в чате, а в файлах на диске!
✅ Результат сдачи: report.csv сформирован, критерии выполнены.
```

Structured Handoff: память проекта живёт на ДИСКЕ в .planning/STATE.md, а каждая фаза работает с максимальной точностью внимания.

ВЫВОД ЧАСТИ 2 // КОНЕЧНЫЙ АВТОМАТ ВМЕСТО БЕСКОНЕЧНОГО ЧАТА

GSD решает проблему распада контекста архитектурно: память проекта вынесена на диск в .planning/, каждый план проверяется скриптом до старта кодирования, а каждая волна изменений выполняется свежим субагентом в чистом окне и передаёт эстафету через 15 строк Structured Handoff.

Часть 2 завершена -> Переходим к практике: Оркестрация и GSD в терминале

ИТОГИ ИНТЕНСИВА // АГЕНТНАЯ ЛЕСТНИЦА PYTEX

Синтез трёх дней: единая система координат профессиональной AI-разработки

Итоги интенсива: Агентная лестница Pytex, дни 1-3

Уровень лестницы	Главный вопрос уровня	Ключевые артефакты в Git	Какой сбой устраняет
День 1. Скилы и Харнес	Как удержать качество одного шага?	AGENTS.md, SKILL.md, pytest	Халтура, пропуск тестов и поломка контрактов
День 2. Конвейер и Панель	Как разделить роли и убрать предвзятость?	specs, .ai-team, git worktree, 3 судьи	Эффект поддакивания (Sycophancy) и самопроверка
День 3. БЗ Карпати	Как ориентироваться в большом проекте?	INDEX.md, domains, verify_kb_sync	Поиск наугад через grep, раздувание окна и амнезия
День 3. Конвейер GSD	Как вести крупную фичу без распада сессии?	.planning: REQUIREMENTS, ROADMAP, STATE, PLAN	Деградация длинного чата и захват чужих модулей

Чек-лист внедрения: 5 шагов в вашем проекте в понедельник

1. Закон репозитория

`AGENTS.md`

Зафиксировать границы слоёв, строгие запреты и Frozen Core.

2. Процедурные скилы

`.agents/skills/`

Упаковать типовые операции в формат L1-L3 (до 400 строк).

3. Ворота качества

`Exit 0`

Настроить ruff, basedpyright и pytest в pre-commit и CI/CD.

4. База знаний

`docs/INDEX.md`

Описать архитектуру проекта и домены компактными карточками.

5. Пофазный GSD

`.planning/`

Разбить большие задачи на изолированные волны с handoff.

Внедряйте инженерный харнес шаг за шагом: начните с AGENTS.md и ворот качества.

ГЛАВНЫЙ ВЫВОД ИНТЕНСИВА

Надёжная разработка с ИИ-агентами - это не поиск «волшебного промпта», а инженерная архитектура внешней памяти и проверок: База знаний Карпати даёт агенту точную карту проекта в 1,5k токенов, а конвейер GSD ведёт разработку короткими изолированными волнами с гарантированной приёмкой.

Спасибо за работу на всех трёх днях интенсива Pytex!

ВОПРОСЫ И ОТВЕТЫ

День 3: База знаний Андрея Карпати и методология GSD: разбор ваших кейсов